

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

The objective of this experiment was to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment used a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. The effect of individual design choices was analyzed using training/validation curves, and a suitable configuration was finally selected using 5-fold cross-validation.

2 Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset.

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images were resized to $224 \times 224 \times 3$.

Experimental considerations followed:

- RGB images were resized to 224×224 .
- Input images were normalized according to the MobileNetV2 pretrained-model requirements.
- Separate training, validation and test data were maintained.
- The test set remained untouched during hyperparameter selection.
- MobileNetV2 was used because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

Table 1: Actual Dataset Split Used

Split	Number of Images
Training	2944
Validation	736
Test	3669

3 MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:

Input ($224 \times 224 \times 3$) $\rightarrow$ Convolution / Depthwise Convolution
 $\rightarrow$ Pointwise Convolution $\rightarrow$ Residual Block
 $\rightarrow$ Global Average Pooling $\rightarrow$ Dense $\rightarrow$ Softmax

Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

Source Code

The complete source code for this experiment is available at: <https://github.com/Jai-Abhishek/DL-Lab>

4 Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

The following initialization strategies were investigated:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Table 2: Weight Initialization – Summary of Results

Initializer	Final Train Loss	Final Train Acc.	Best Val. Accuracy (%)
Xavier/Glorot	0.0071	1.0000	91.58
He	0.0065	1.0000	91.30
Random	0.0123	1.0000	90.90
Zero	3.6104	0.0279	1.77

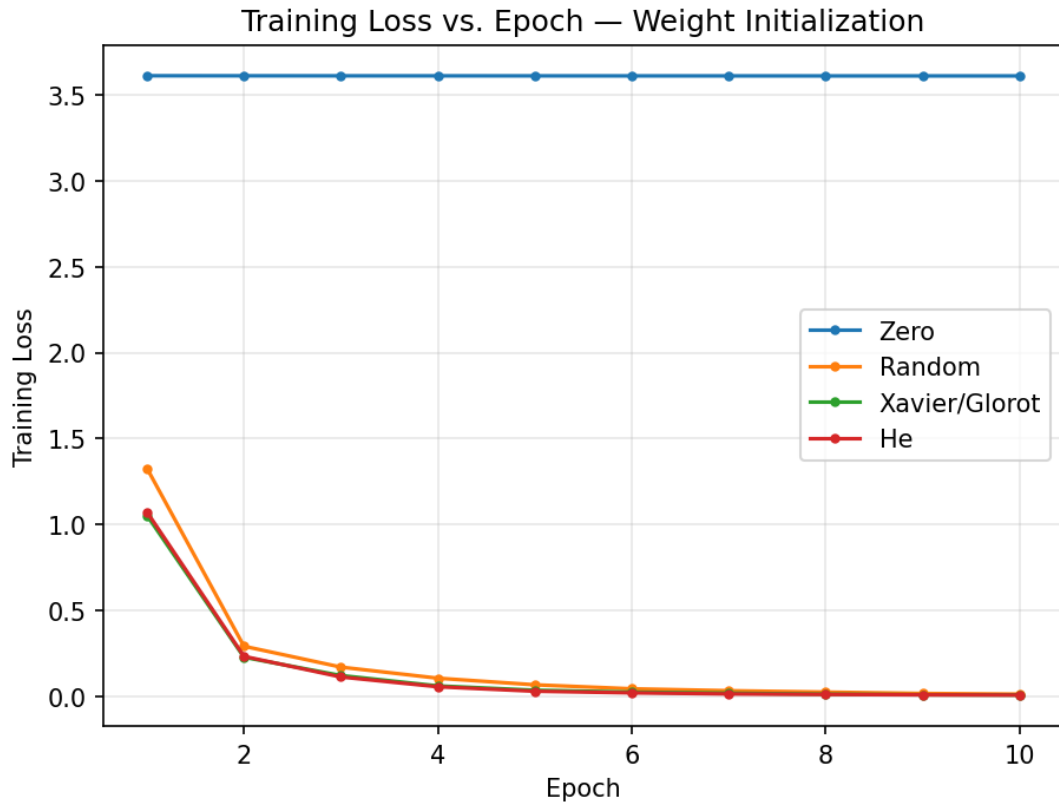


Figure 1: Training Loss vs. Epoch for Different Weight Initialization Strategies

Inference

- Figure 1 tracks how the training loss evolves across 10 epochs for the four initialization schemes – Zero, Random, Xavier/Glorot and He.
- Both Xavier/Glorot and He drove the loss down the quickest, settling near 0.007 after only a handful of epochs.
- Random initialization needed a few more epochs to catch up, but eventually landed at a similarly low loss value.
- Zero initialization, by contrast, stayed pinned at roughly 3.61 for the entire run – a clear sign that no useful learning was taking place.
- That stagnation matches the well-known symmetry issue with all-zero weights: every neuron in a layer receives the same gradient update, so none of them ever specialize.

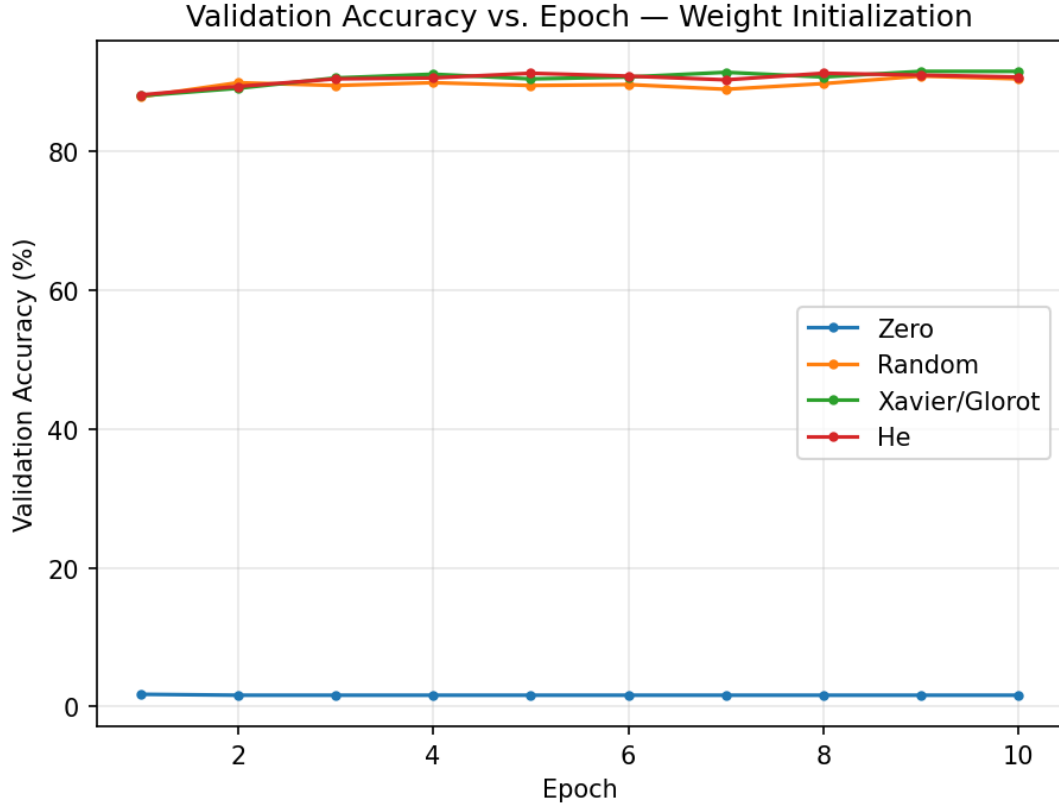


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initialization Strategies

- Validation accuracy tells a similar story: Xavier/Glorot came out on top at 91.58%, with He (91.30%) and Random (90.90%) not far behind.
- Zero initialization never really got going, sitting at about 1.77% – barely better than guessing among 37 breed classes.
- The wide gap between Zero and the other three schemes shows that the classifier head genuinely needed a properly scaled starting distribution to learn anything useful from the frozen MobileNetV2 features.

5 Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

The following configurations were compared:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Table 3: Regularization – Final Epoch Metrics and Generalization Gap

Configuration	Train Acc.	Val Acc.	Gap	Train Loss	Val Loss
No Regularization	1.0000	0.9103	0.0897	0.0074	0.2702
L2 Regularization	1.0000	0.9103	0.0897	0.2038	0.4348
Dropout	0.9630	0.9063	0.0567	0.1273	0.2521
Batch Normalization	0.9993	0.9063	0.0931	0.0070	0.3140

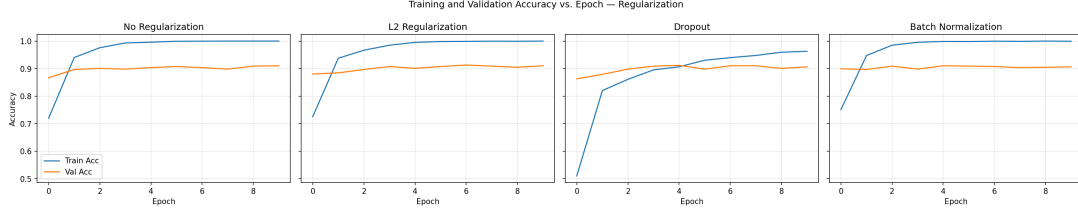


Figure 3: Training and Validation Accuracy vs. Epoch for Different Regularization Strategies

- Both No Regularization and L2 Regularization drove training accuracy all the way to 1.0, even though validation accuracy hovered around 0.91 for every configuration.
- Dropout kept the train-validation gap the tightest of the four (0.0567), largely because its training accuracy never quite reached 1.0 over the 10 epochs.
- Batch Normalization got very close to perfect training accuracy (0.9993) without a matching jump in validation accuracy, which left it with the widest generalization gap of the group (0.0931).

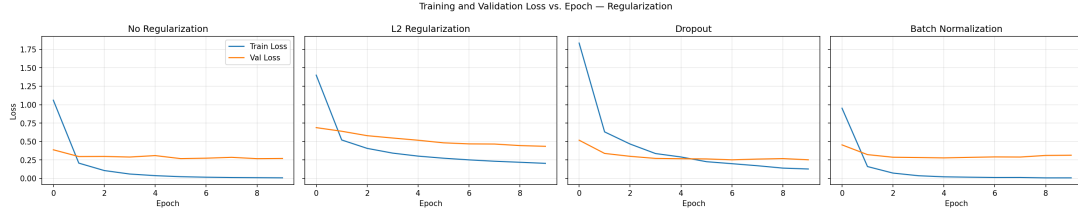


Figure 4: Training and Validation Loss vs. Epoch for Different Regularization Strategies

- None of the four setups showed validation loss climbing steadily over the 10 epochs, though training loss dropped well below validation loss everywhere except under Dropout.
- L2 Regularization's training loss stayed noticeably higher (0.2038) than the rest, which makes sense since the L2 penalty is added directly into the loss function being optimized.
- Dropout's training and validation loss curves tracked each other most closely of the four, which lines up with the smaller generalization gap it showed in Table 3.

6 Batch Normalization

With Batch Normalization vs. Without Batch Normalization

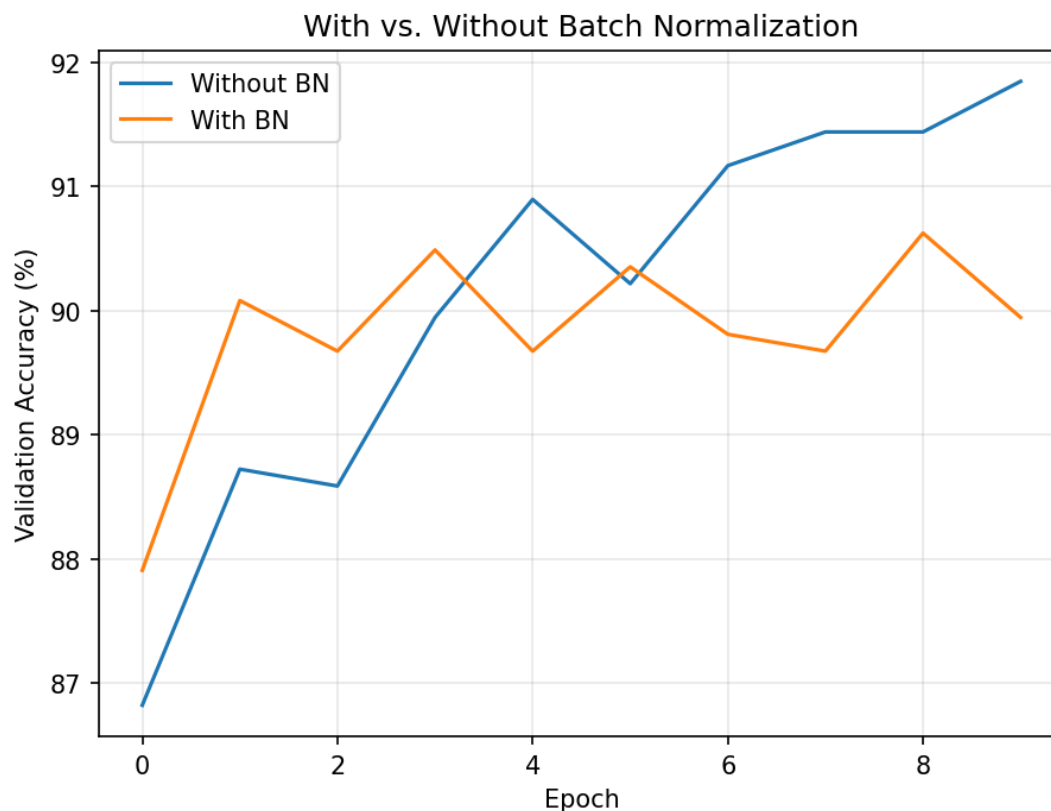


Figure 5: Validation Accuracy vs. Epoch – With BN vs. Without BN

- Figure 5 follows validation accuracy over 10 epochs for a classifier head trained with and without a Batch Normalization layer.
- The Without-BN run started off slower (86.8%) but had overtaken the With-BN run by epoch 10, finishing at 91.85%.
- With-BN climbed faster early on – 90.1% by epoch 2 – but then leveled off, oscillating between roughly 89.7% and 90.6% for the rest of training.
- In this run, then, adding Batch Normalization on top of an already-normalized, frozen MobileNetV2 feature vector did not give the single-layer classifier head any lasting accuracy advantage over leaving it out.

7 Optimization Algorithms

The following optimization methods were compared: SGD, Momentum, RMSProp and Adam. The objective was to study convergence speed, stability and final validation performance.

Table 4: Optimizer Comparison

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	2.3140	0.5109	10	99.82
Momentum	0.2469	0.9117	6	105.93
RMSProp	0.0029	0.9144	5	108.77
Adam	0.0070	0.9158	4	102.73

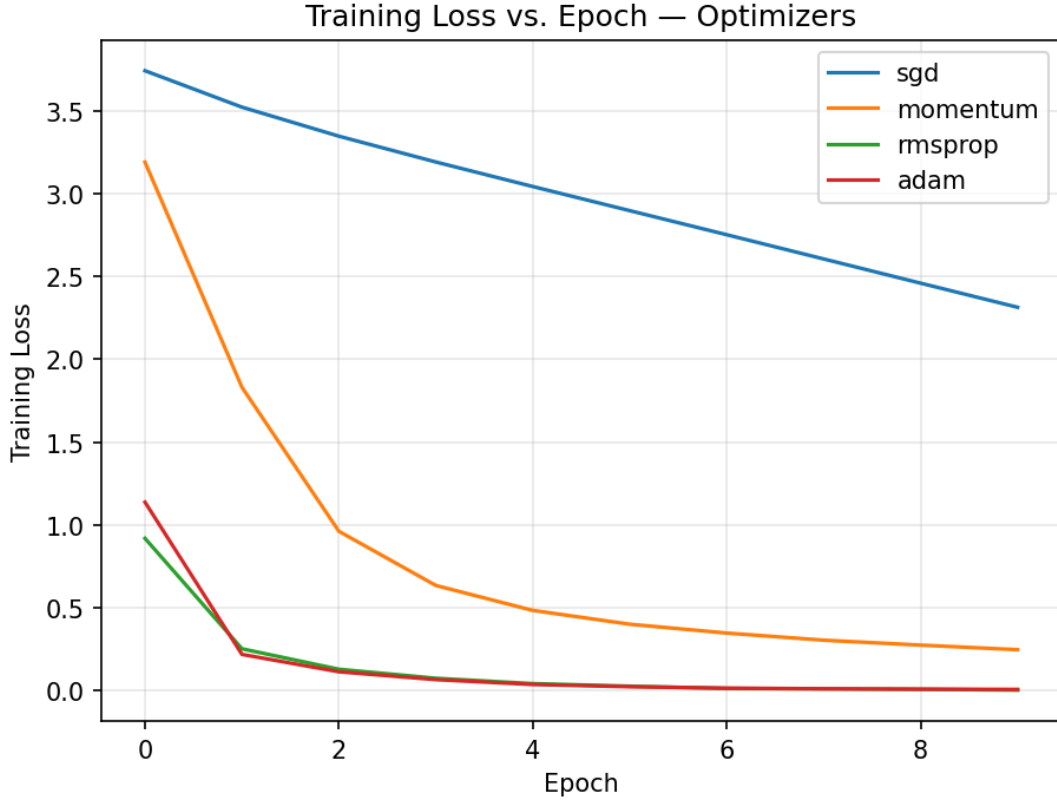


Figure 6: Training Loss vs. Epoch for Different Optimizers

- Plain SGD was clearly the slowest to bring the loss down, finishing at 2.3140.
- Both RMSProp and Adam pushed training loss under 0.01 well within the epoch budget.
- SGD's sluggish progress fits with how it works: a single fixed learning rate applied uniformly to every parameter, with no momentum term or adaptive scaling to speed things along.

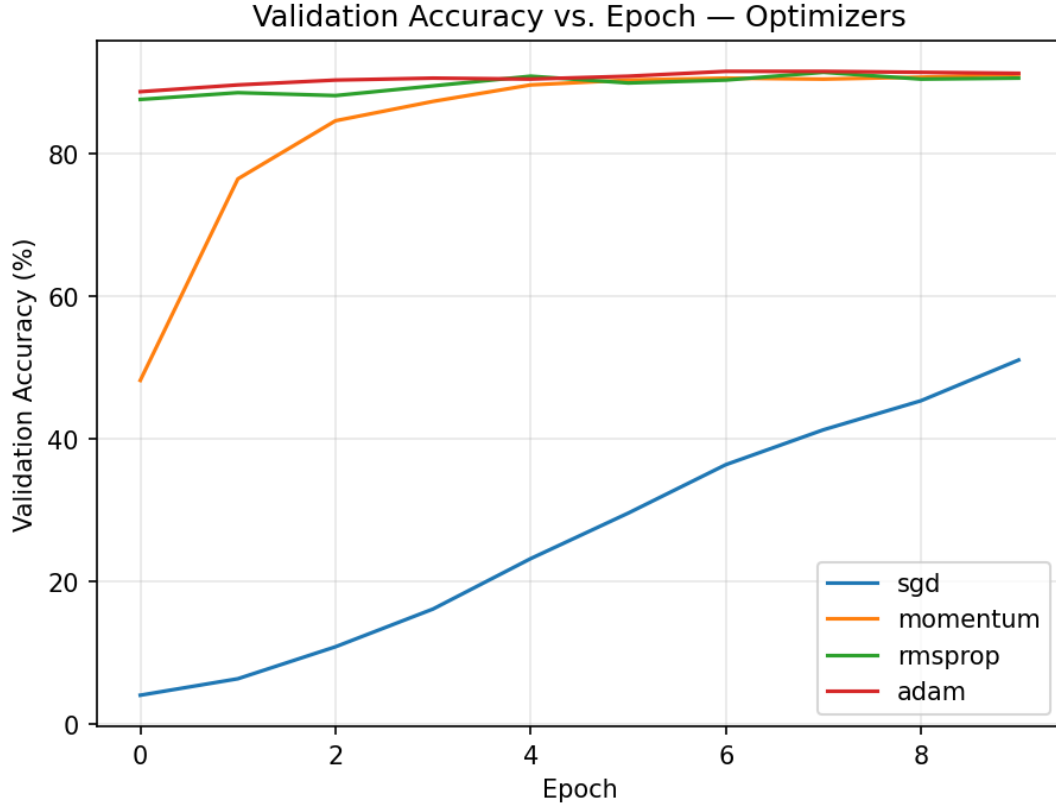


Figure 7: Validation Accuracy vs. Epoch for Different Optimizers

- Adam came out ahead on best validation accuracy (91.58%), with RMSProp (91.44%) and Momentum (91.17%) close behind.
- Plain SGD lagged well behind the rest, topping out at 51.09% over the 10 trained epochs.
- Adam also got there fastest, needing only 4 epochs to converge.
- Despite these differences in accuracy and convergence speed, wall-clock training time stayed roughly the same across all four optimizers, in the 100–109 second range.

8 CNN Hyperparameter Tuning

The following hyperparameters were investigated using a controlled one-variable-at-a-time methodology, with baseline values of learning rate = 0.001, batch size = 32, dropout = 0, optimizer = Adam.

Table 5: Hyperparameter Candidate Values

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

Table 6: Hyperparameter Sweep – Best Validation Accuracy

Setting	Best Validation Accuracy (%)
Learning Rate = 0.001	91.30
Learning Rate = 0.0001	91.03
Batch Size = 16	91.85
Batch Size = 32	92.39
Batch Size = 64	91.30
Dropout Rate = 0.00	90.90
Dropout Rate = 0.25	92.26
Dropout Rate = 0.50	91.71

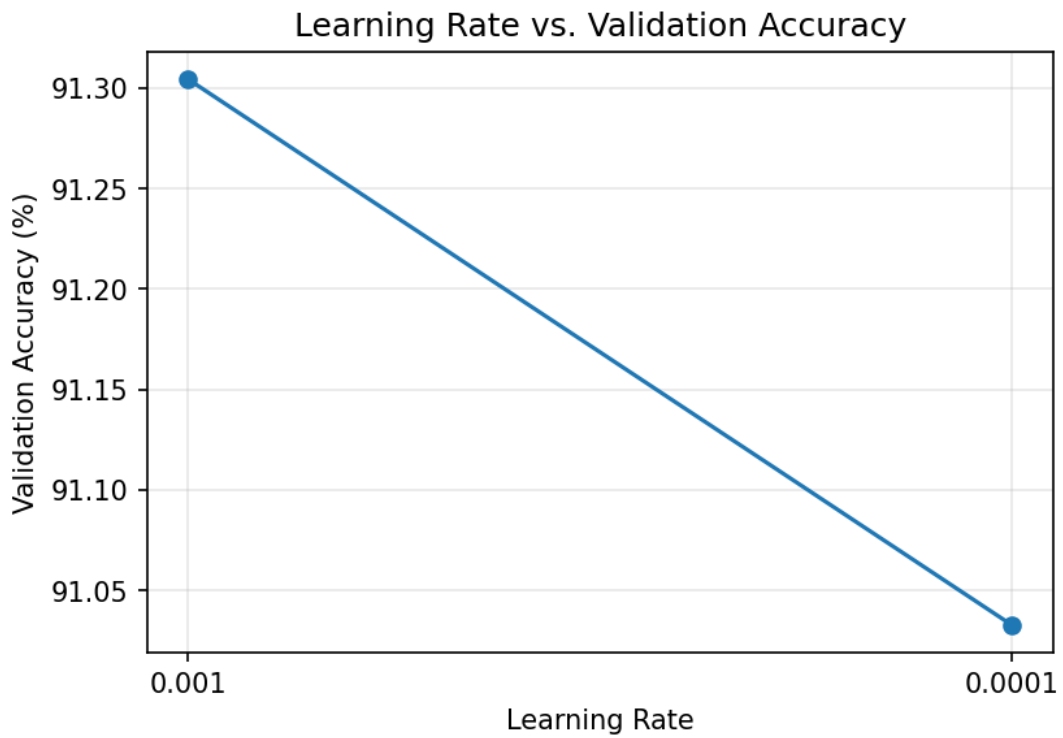


Figure 8: Learning Rate vs. Validation Accuracy

- The larger learning rate (0.001) edged out the smaller one, reaching 91.30% best validation accuracy versus 91.03% for 0.0001.
- Since the gap between the two is so small, validation accuracy does not appear to be very sensitive to this particular learning-rate change within the epoch budget used.

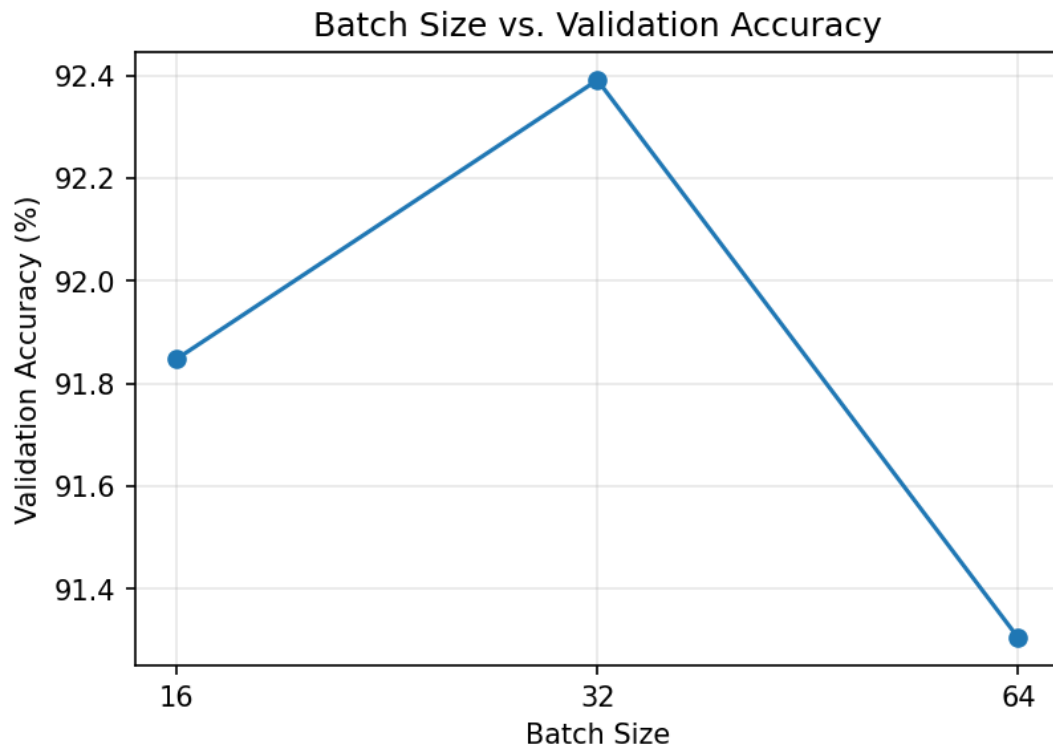


Figure 9: Batch Size vs. Validation Accuracy

- A batch size of 32 gave the best result of the three, reaching 92.39% validation accuracy.
- The dip at batch size 64 could simply be a consequence of fewer gradient updates happening per epoch as the batch grows larger, while the number of epochs stayed fixed.

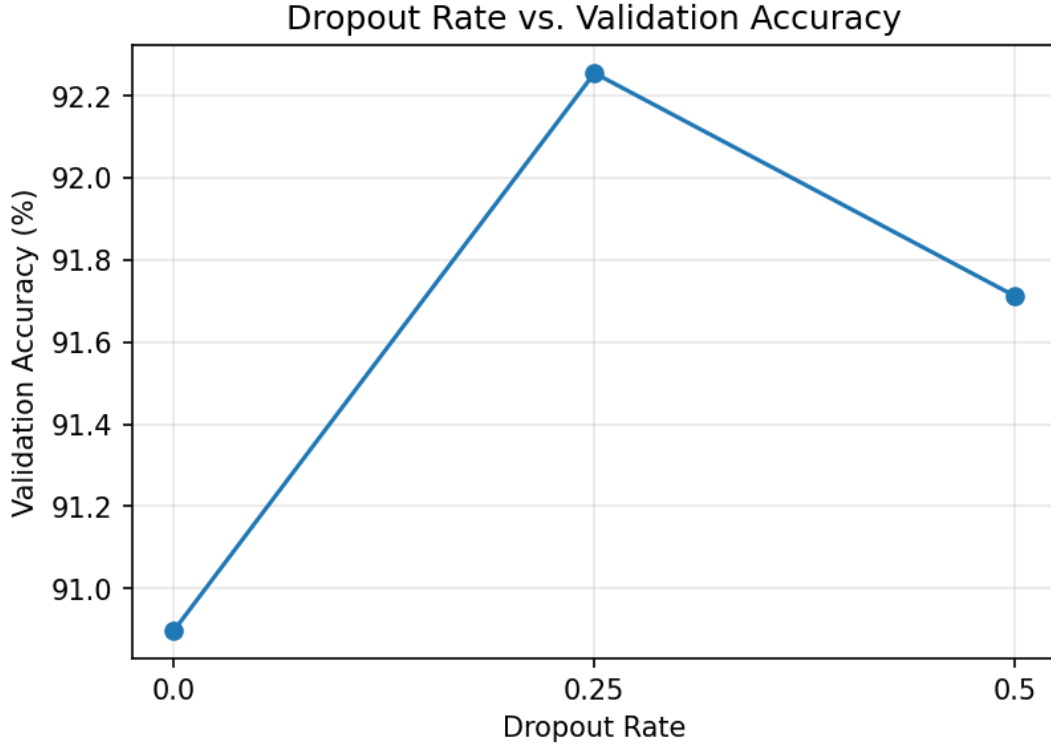


Figure 10: Dropout Rate vs. Validation Accuracy

- A moderate dropout rate of 0.25 came out on top (92.26%), ahead of both 0.5 (91.71%) and no dropout at all (90.90%).
- This middle-of-the-road result fits the usual pattern for dropout as a regularizer: too little of it gives limited benefit, too much starts to hurt learning, and a moderate value tends to work best.

9 Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet was used as the CNN backbone for all experiments in this report.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers were frozen and only the newly added classification layers were trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network (approximately the final third of the backbone) were unfrozen and trained together with the classifier head, using a smaller learning rate (1×10^{-4}) than the feature-extraction stage.

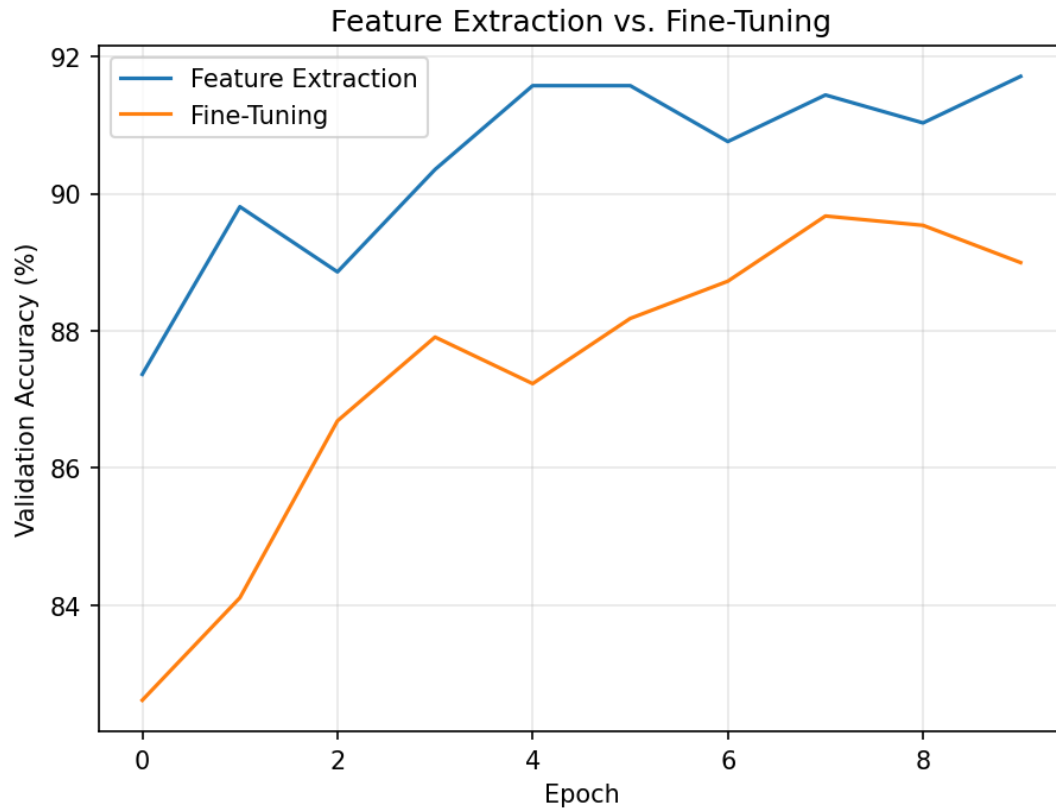


Figure 11: Feature Extraction vs. Fine-Tuning – Validation Accuracy

- Feature Extraction stayed ahead on validation accuracy throughout training, finishing around 91.7% at epoch 10, compared with roughly 89.0% for Fine-Tuning.
- Fine-Tuning began much lower, around 82.6% at epoch 1, and climbed steadily thereafter, but it had not closed the gap by the end of the 10-epoch run.

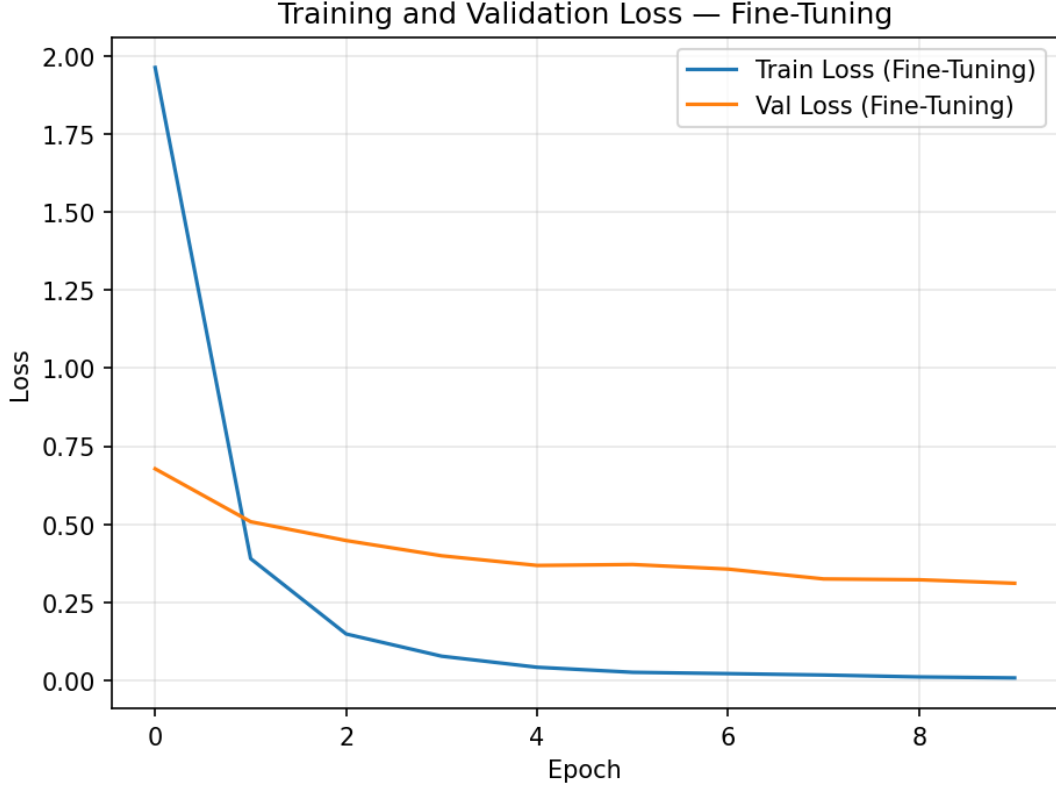


Figure 12: Training and Validation Loss – Fine-Tuning Stage

- Training and validation loss both trended downward across the plotted epochs, with no sign of the two curves pulling apart.
- Since validation loss was still falling at the end of the run, Fine-Tuning likely had not yet hit its ceiling – more epochs may have pushed it further.

Why Fine-Tuning Uses a Smaller Learning Rate

- The unfrozen layers already contain useful ImageNet-pretrained weights.
- A small learning rate limits how much these pretrained weights are perturbed per update.
- A large learning rate can cause large destructive updates and effectively discard the benefit of transfer learning.

10 K-Fold Cross-Validation

Four candidate configurations, selected from the preceding individual studies, were evaluated using 5-fold cross-validation on the training data. The independent test set was not used during this stage.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}$$

The result is reported as $\bar{A} \pm SD$.

Table 7: 5-Fold Cross-Validation Results

Configuration	F1	F2	F3	F4	F5	Mean	SD
C1 baseline adam	0.9253	0.9063	0.9103	0.9035	0.9076	0.9106	0.0077
C2 dropout adam	0.9171	0.9049	0.9185	0.8927	0.9130	0.9092	0.0095
C3 bn adam	0.9063	0.9090	0.9130	0.9076	0.9158	0.9103	0.0035
C4 l2 rmsprop	0.9076	0.9022	0.9198	0.8954	0.8967	0.9043	0.0089

Here, C1 = baseline (no regularization, Adam), C2 = Dropout (0.5, Adam), C3 = Batch Normalization (Adam), and C4 = L2 regularization with RMSProp.

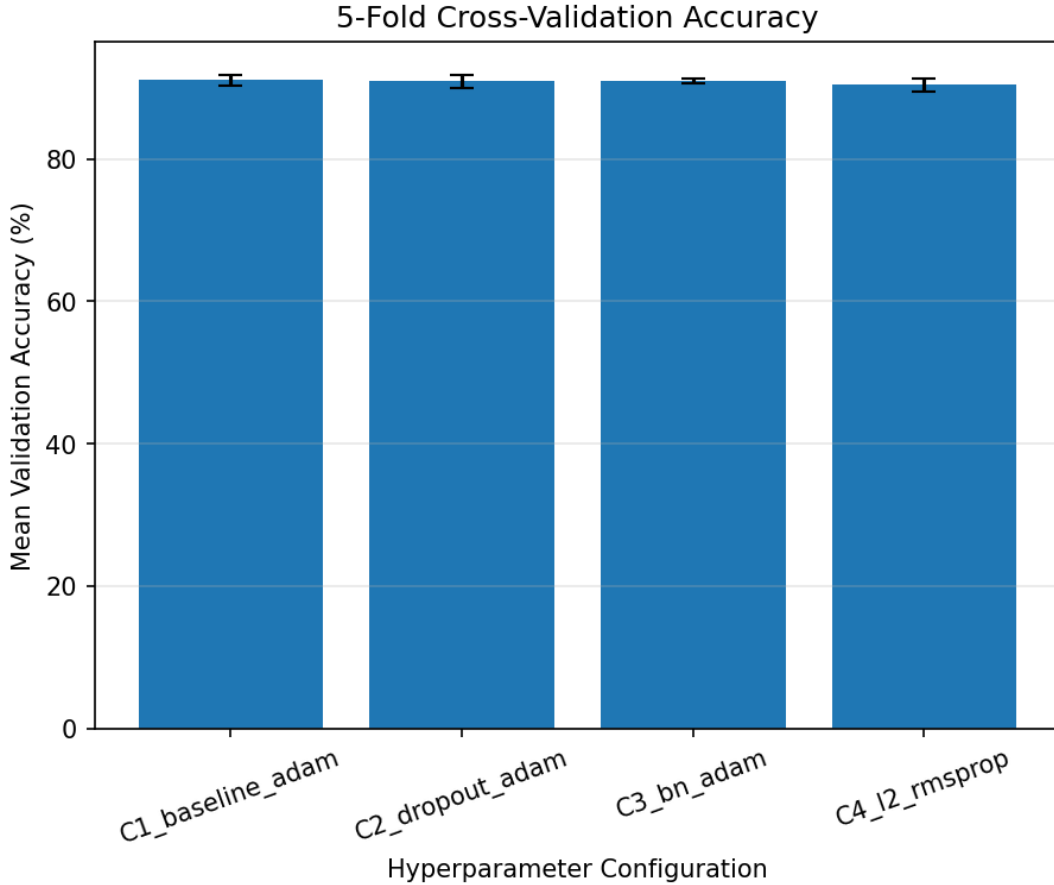


Figure 13: 5-Fold Cross-Validation Accuracy with Standard-Deviation Error Bars

- C1 (baseline, Adam) posted the highest mean accuracy (0.9106), with C3 BN Adam (0.9103) and C2 Dropout Adam (0.9092) close behind; C4 L2 RMSProp trailed the group (0.9043).
- C3 BN Adam was the most consistent configuration, with the lowest standard deviation across folds (0.0035).
- Because C1 combined the top mean accuracy with a reasonably low standard deviation (0.0077), it was the configuration carried forward to final model evaluation.

11 Final Model Evaluation

The C1 baseline Adam configuration, selected by 5-fold cross-validation, was retrained using the complete training data. The independent test set was kept untouched until this stage and was then used for a single final evaluation.

Table 8: Final Model Evaluation

Metric	Value
Mean CV Accuracy	0.9106
CV Standard Deviation	0.0077
Test Accuracy	0.8948
Precision (macro)	0.8955
Recall (macro)	0.8940
F1-score (macro)	0.8936
Training Time (s)	105.49
Number of Parameters	2,426,725

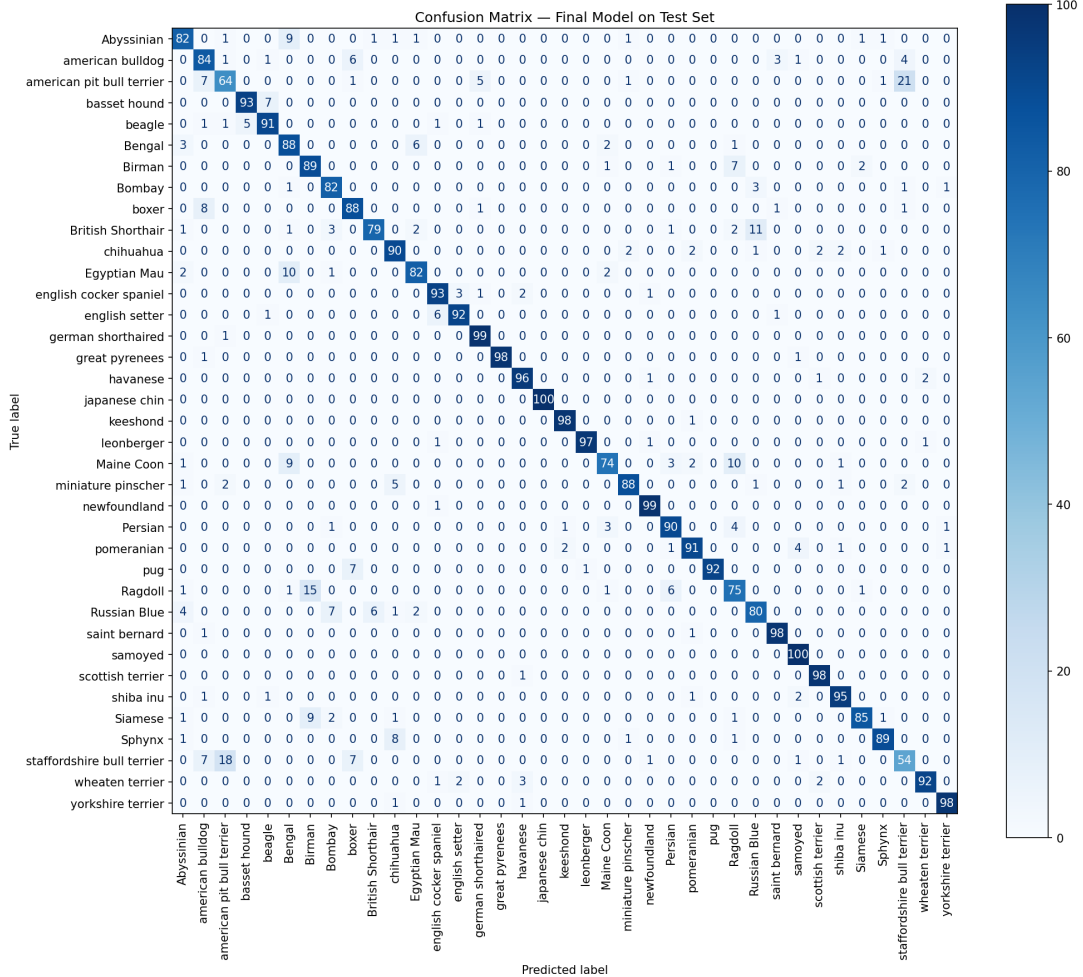


Figure 14: Confusion Matrix – Final Model on Test Set

- The model handled several breeds almost perfectly, including Japanese Chin (100), Samoyed (100), German Short-haired (99), Newfoundland (99), Great Pyrenees (98), Keeshond (98), Saint Bernard (98), Scottish Terrier (98) and Yorkshire Terrier (98).

- Its biggest weak spot was distinguishing Staffordshire Bull Terrier from American Bulldog: 18 Staffordshire Bull Terrier images ended up labeled as American Bulldog.
- Similar breed-pair confusions showed up elsewhere too – American Pit Bull Terrier was frequently mistaken for Staffordshire Bull Terrier (21 images), and Ragdoll for Birman (15 images).

12 Overall Results

Table 9: Overall Results Across Experiment Stages

Configuration	Best Validation Accuracy (%)	Test Accuracy (%)
Best Initialization	91.58	–
Best Regularization	91.30	–
Best Optimizer	91.58	–
Best Hyperparameters	92.39	–
Fine-Tuned Model	89.67	–
Final Selected Model	91.06 (CV Mean)	89.48

- Across all the stages studied, the Best Hyperparameters setup (batch size 32, dropout 0.25) produced the single highest validation accuracy of the entire experiment (92.39%).
- Fine-tuning did not outperform simple feature extraction here, landing at a lower validation accuracy (89.67%) within the epochs allotted.
- On the held-out test set, the final selected model scored a bit lower (89.48%) than its cross-validation mean (91.06%), which is a fairly typical drop when moving from CV to a genuinely unseen test set.

13 Discussion Questions

1. **Difference Between Parameters and Hyperparameters:** Parameters (weights and biases) are what the model learns during training, whereas hyperparameters – learning rate, batch size, dropout rate, and similar settings – are chosen beforehand and control how that learning happens.
2. **Why Is Weight Initialization Important?** The starting values of the weights shape how training unfolds from the very first step, influencing both how fast the model converges and how well it ultimately performs.
3. **Why Can Zero Initialization Be Problematic for Neural Networks?** When every weight starts at zero, all neurons in a layer compute identical outputs and receive identical gradients, so they never break symmetry – exactly what happened here, where the model was stuck at only 1.77% validation accuracy.
4. **Compare Xavier and He Initialization:** Xavier/Glorot initialization is tuned for activations like sigmoid and tanh, while He initialization is calibrated specifically for ReLU-family activations.
5. **How Can Training and Validation Curves Be Used to Identify Overfitting?** A telltale sign of overfitting is training performance continuing to climb while validation performance flattens out or starts getting worse.

6. **How Does Dropout Reduce Overfitting?** By randomly switching off a fraction of neurons on each training step, Dropout stops the network from leaning too heavily on any single neuron, which helps it generalize better.
7. **What Is the Purpose of Batch Normalization?** Batch Normalization re-centers and re-scales layer activations during training, which tends to make the optimization process smoother and more stable.
8. **Explain the Numerical Batch Normalization Example:** Taking $x = [2, 4, 6, 8]$, the batch mean works out to 5 and the variance to 5; normalizing each value accordingly gives roughly $[-1.342, -0.447, 0.447, 1.342]$.
9. **What Are the Roles of γ and β ?** After normalization, γ rescales the values and β shifts them, giving the network the freedom to recover whatever scale and offset actually works best for that layer.
10. **Compare SGD, Momentum, RMSProp and Adam:** Plain SGD steps using only the current gradient; Momentum folds in a running average of past gradients to smooth the path; RMSProp adapts the effective learning rate per parameter based on recent gradient magnitudes; and Adam combines both the momentum idea and the per-parameter adaptive scaling.
11. **What Happens When the Learning Rate Is Too Large?** Steps become too aggressive, the optimizer can overshoot minima, and training loss may oscillate or diverge outright instead of decreasing.
12. **What Happens When the Learning Rate Is Too Small?** Progress becomes very slow, and the model may fail to reach good performance within the epochs available for training.
13. **What Is the Effect of Increasing Batch Size?** Larger batches produce smoother, lower-variance gradient estimates but also mean fewer weight updates per epoch; in this experiment, a batch size of 32 struck the best balance.
14. **Explain Stride and Padding:** Stride sets how many pixels the convolution filter shifts between applications, while padding adds extra border pixels to the input so the output size can be controlled independently of stride.
15. **Why Is MobileNetV2 Computationally Efficient?** It relies on depthwise separable convolutions and inverted residual blocks, both of which cut down substantially on the number of computations and parameters compared to standard convolutions.
16. **What Is Depthwise Separable Convolution?** It splits an ordinary convolution into two lighter steps: a depthwise convolution applied per input channel, followed by a 1×1 pointwise convolution that combines the channels – together far cheaper than a standard convolution.
17. **What Is Transfer Learning?** Transfer learning reuses the representations already learned by a model trained on one task – here, MobileNetV2 pretrained on ImageNet – as a starting point for a different, related task.
18. **Differentiate Feature Extraction and Fine-Tuning:** In feature extraction, the pretrained layers stay frozen and only new layers are trained; in fine-tuning, some of those pretrained layers are unfrozen and updated along with the new layers.
19. **Why Is a Smaller Learning Rate Generally Used During Fine-Tuning?** Keeping the learning rate small limits how much the already-useful pretrained weights are disturbed by each update, avoiding the risk of erasing what they pre-

viously learned.

20. **Why Is K-Fold Cross-Validation Useful for Hyperparameter Selection?** By evaluating a configuration across several different train/validation splits rather than just one, K-Fold cross-validation gives a more reliable, less split-dependent estimate of how well it performs.
21. **Why Must the Test Set Remain Untouched During Tuning?** Keeping the test set completely separate from any tuning decisions ensures it still gives an honest, unbiased read on the final model’s real-world performance.
22. **Why Should Mean and Standard Deviation Both Be Reported?** The mean captures typical performance across folds, while the standard deviation captures how much that performance varies – both matter for judging a configuration fairly.
23. **Is the Highest Validation Accuracy Always Sufficient to Select a Model?** Not necessarily – factors like consistency across folds, generalization to unseen data, and other practical constraints also need to be weighed alongside raw accuracy.

14 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation.
6. Keras Documentation.